

M1 Algebraic Algorithms for Cryptology: Homework 2

Hatim Ridaoui

April 2025

Exercise 1 – Reed–Solomon Code over $K = \mathbb{Z}/65537\mathbb{Z}$

Throughout this exercise let $K = \mathbb{Z}/65537\mathbb{Z}$ be the prime field.

1.a Definition of an (n, k) Reed–Solomon code over K

Let n, k be integers with $1 \leq k \leq n \leq q$. Let's have an ordered set of n distinct evaluation points

$$A = (\alpha_1, \dots, \alpha_n) \subset K.$$

The Reed–Solomon code of length n and dimension k over K (with evaluation set A) is

$$\text{RS}_K(A, k) := \left\{ (m(\alpha_1), \dots, m(\alpha_n)) \mid m \in K[x], \deg m < k \right\} \subset K^n.$$

Each codeword is just the list of the polynomial's values at the chosen points.

1.b Maximum length and error-correction capability

The α_i must be pairwise distinct, so the longest possible code over K has length

$$n_{\max} = |K| = 65537.$$

A Reed–Solomon code is MDS, so its minimum Hamming distance is $d = n - k + 1$. So a decoder can correct up to

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor = \left\lfloor \frac{n-k}{2} \right\rfloor$$

errors in any received word.

1.c Encoder (parameters n and k)

We implemented an encoder in `rs_encoder.c`. It supports any $1 \leq k \leq n \leq 65537$ and has two unit tests in `rs_encoder_test.c`: a fixed golden vector and a randomized identity check over 500 trials. (Plus simple testing in the file itself). Both tests pass.

See code files

1.d Decoder based on the Extended Euclidean Algorithm

The EE-algorithm decoder is functional for most blocks but still fails. What we tried to implement, following the Extended Euclidean Algorithm :

1. **Interpolation:** use Lagrange to obtain the received polynomial $R(x)$ from the n pairs (α_i, r_i) .
2. **Vanishing polynomial:** build $V(x) = \prod_{i=1}^n (x - \alpha_i) = x^n - 1$.
3. **EE step:** run the EEA on (V, R) until the remainder $r(x)$ has degree $< k + t$. The algorithm returns polynomials r, v such that $r = vR \bmod V$.
4. **Message recovery:** divide $m(x) = r(x)/v(x)$ (expected to be exact with $\deg m < k$) and re-encode it to obtain the corrected code-word.

I think that the current implementation passes all random blocks with up to $t - 1$ errors but still miscounts the error weight when *all* t error positions produce a zero magnitude. Maybe the cause is the normalisation of $v(x)$.

See code files

1.e Demonstration on sample files

We can already encode text files (see `Alice.txt`), inject noise with the symmetric q -ary channel and report the symbol-error statistics (`ex1f_demo`). Once the decoder works, this demo will turn into a full end-to-end test.

See code files

Exercise 2 – Reed–Solomon code over $\text{GF}(2^7)$

- Galois $\mathbb{F}_{128}$ field layer from Homework 1 reused; two back-ends (bit-serial and Zech tables).
- Encoder ported (`rs128_encoder.c`) and tested; identity and golden-vector tests pass.
- Decoder is pending.

See code files